Inheritance

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya)

Classes II

- java.lang.Object is the **supertype** of all Java classes
- equals(), toString(), and hashCode() have a **default implementation based on memory addresses**, but new classes can provide their own implementation
- clone() implemented in the class Object provides a **shallow copy** of the instance
- It is a bad idea to use mutable object as keys in Hash-code based collections
- It is a good idea to reuse existing classes, e.g., java.lang.String, java.time.LocalDate, java.util.ArrayList

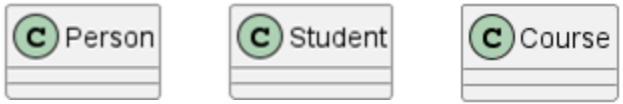
Motivation

- Sometimes we might need the same code in multiple classes
- Inheritance facilitates **code reuse** and **customization**
- New classes (subclass) can **inherit** properties and behavior from existing classes (superclass)

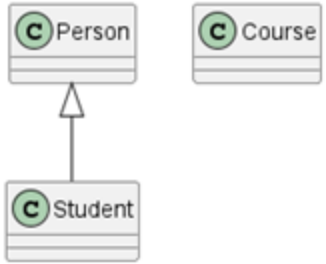
Requirement

- There must be a **“is-a” relationship** between subclass and superclass
  - **A is a subclass of B only if any instance of A is also an instance of B**

Example



A is a subclass of B only if any instance of A is also an instance of B



Any student is also a person

Student is a subclass of Person

```
class Student extends Person {
    ...
}
```

- Person is the superclass / base class / parent class of Student
- Student is a subclass / derived class / child class of Person

How many constructors does the class Student have?

```
class Person {

    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

```
class Student extends Person {

}
```

a) 0
b) 1
c) 2
d) More than 2

Please provide your answers using this form

How many constructors does the class Student have?

```
class Person {

    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

```
class Student extends Person {

}
```

a) 0
b) 1
c) 2
d) More than 2

How many instance methods can we use on an instance of the class Student?

```
class Person {

    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

```
class Student extends Person {

}
```

a) 0
b) 1
c) 2
d) More than 2

How many instance methods can we use on an instance of the class Student?

```
class Person {

    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

```
class Student extends Person {

}
```

a) 0
b) 1
c) 2
d) More than 2

All Java classes are subclasses of Object

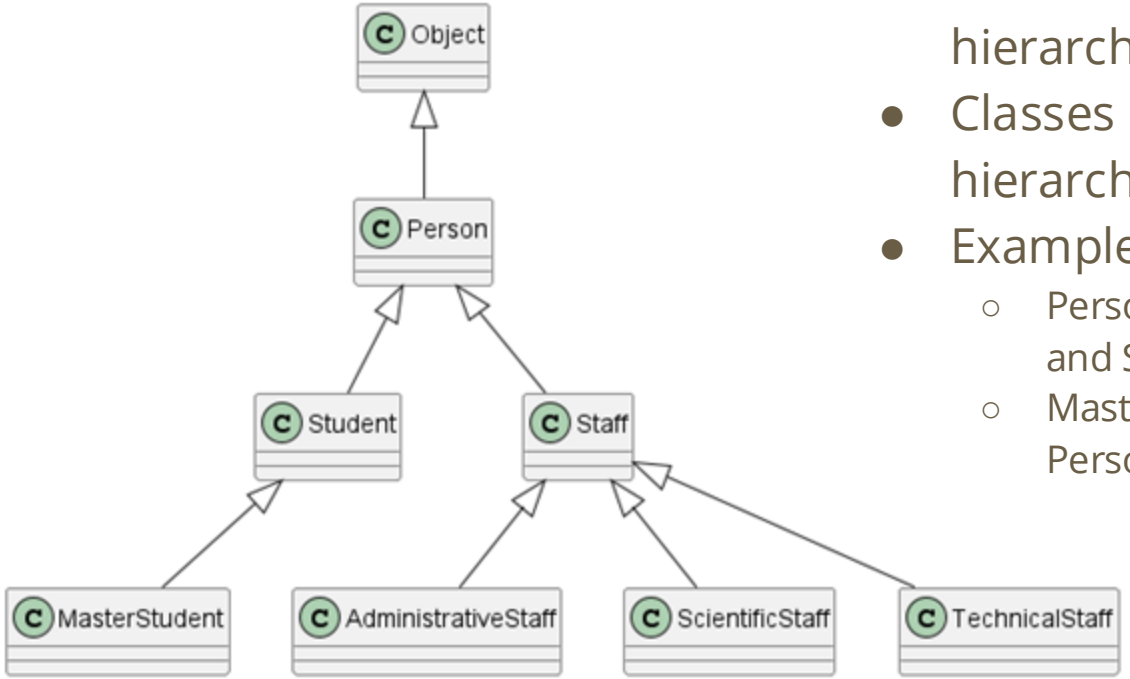
```
class Person {

    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

```
class Person extends Object {

    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Implicitly, Person has Object as its superclass

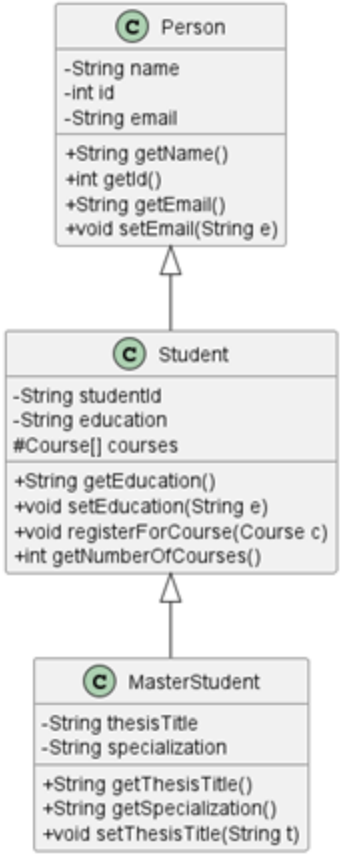
Making its superclass explicit

Hierarchical Relationship



- Classes in higher levels of the hierarchy are called **ancestors**
- Classes in lower levels of the hierarchy are called **descendants**
- Examples:
  - Person is an ancestor of MasterStudent and ScientificStaff
  - MasterStudent is a descendant of Person and Object

Inheriting class members



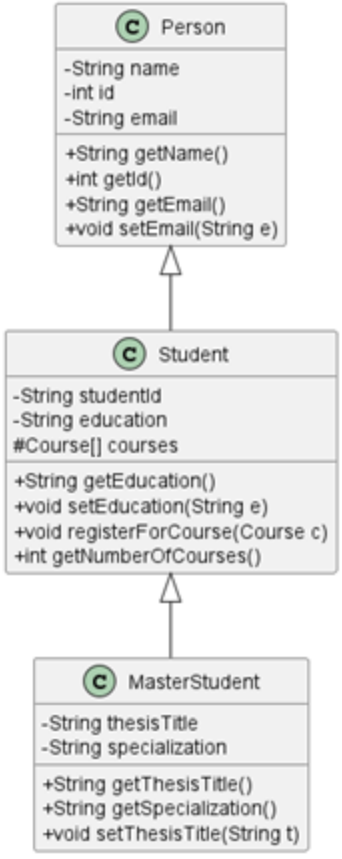
Possible access level for class members:

- **public** (**+**): accessible everywhere the class is accessible
- **protected** (**#**): from the same package or *a **descendant*** of the class
- default or package-level (**~**, no modifier)
  - Typical use: helper classes
- **private** (**-**): accessible only within the body of the class

- Subclasses *inherit* non-private members (fields / methods) of its superclass
  - Package-level members are only inherited if subclass and superclass are part of the same package
  - Subclasses could also *use* private members, e.g., instance fields, by means of an inherited method

The implementation of `getSpecialization` (in the class `MasterStudent`) can access `courses` directly (member of the class `Student`)

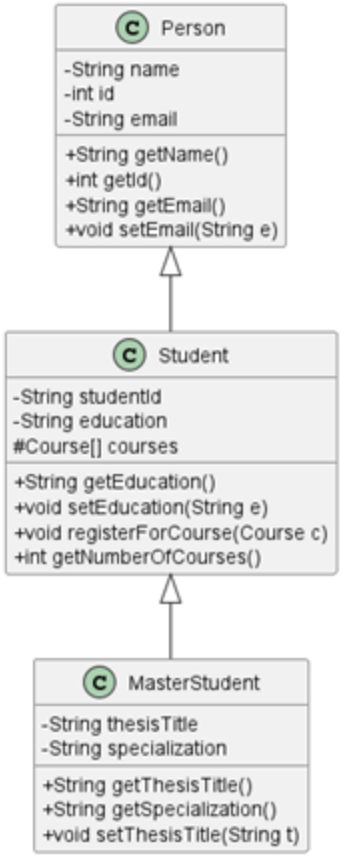
How many methods does Student inherit from Person?



a) 0
b) 4
c) 8
d) More than 8

Please provide your answers using this form

How many methods does Student inherit from Person?

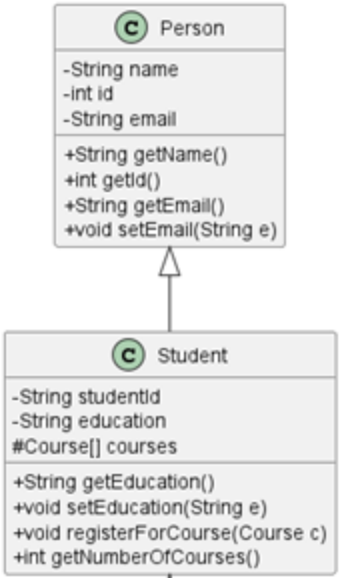


a) 0
b) 4
c) 8
d) More than 8

- getClass(), notify(), notifyAll(), wait(), toString(), equals(), hashCode(), clone(), finalize()
- getName(), getId(), getEmail(), setEmail()

Upcasting

Upcasting: assigning an instance of the subclass to a variable with supertype type (it is always possible)



Any instance of Student **is also** an instance of Person

```
Person p = new Person("Jens", 123);

p.setEmail("jens24@stu.unix.dk");

String welcomeMsg = "Welcome "+p.getName()+"!";
```

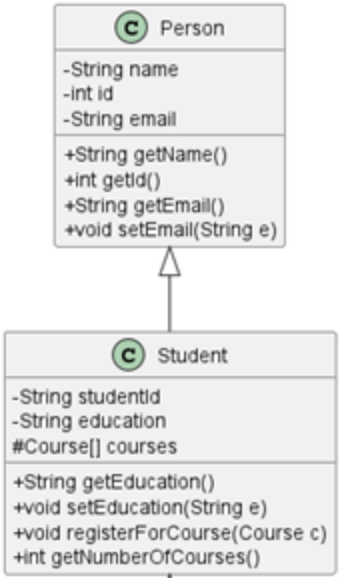
The code that works for an instance of the superclass, **continues to work** for an instance of the subclass

Any instance of Student **must behave** as an instance of Person

```
Person p = new Student("Jens", 123, "jens24");

p.setEmail("jens24@stu.unix.dk");

String welcomeMsg = "Welcome "+p.getName()+"!";
```

Downcasting

Downcasting: assigning an instance of the superclass to a variable with subtype type (it is **not** always possible)



Some instances of Person **are also** instances of Student

```
Student s = new Student("Jens", 123, "jens24");

s.setEducation("Computer Science");

s.registerForCourse(c1);
```

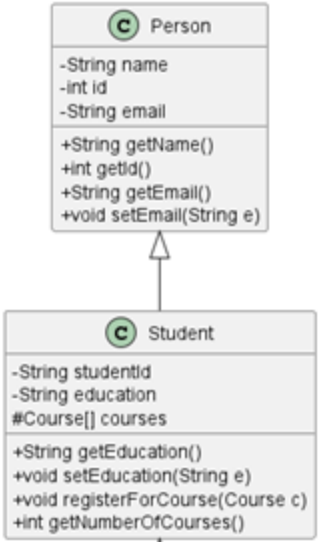
The code that works for an instance of the subclass, **does not continue to work** for any instance of the superclass

```
Student s = new Person("Jens", 123);
```

```
Person p = new Person("Jens", 123);

p.setEducation("Computer Science");

p.registerForCourse(c1);
```

Downcasting

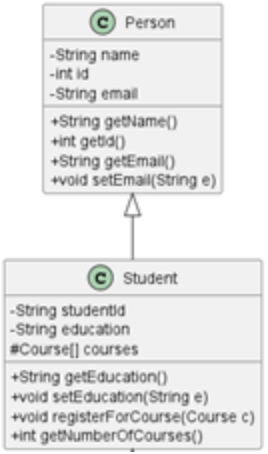
```
Course c1 = new Course("OOP", "Object-oriented programming");

Person p = new Student("Jens", 123, "jens24");

p.setEmail("jens24@stu.unix.dk");

String welcomeMsg = "Welcome "+p.getName()+"!";

Student s = (Student) p;

s.setEducation("Computer Science");

s.registerForCourse(c1);
```



A **typecast** allows for assigning an instance of the supertype to a variable with subtype type (downcasting)

```
T x = (V) y; // V must be T or a subtype of T
```

What is true about the following code?




```
Course c1 = new Course("OOP",
        "Object-oriented programming");

Person p = new Person("Jens", 123);

Student s = (Student) p;

s.setEducation("Computer Science");

s.registerForCourse(c1);

System.out.println("Done");
```

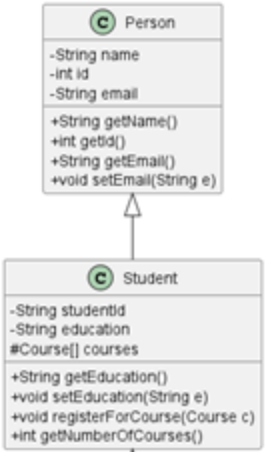
a) The code doesn't compile
b) The code compiles but its execution never reaches the last statement
c) The code compiles and its execution completes without any problems
d) None of the above

A **typecast** allows for assigning an instance of the supertype to a variable with subtype type (downcasting)

```
T x = (V) y; // V must be T or a subtype of T
```

Please provide your answers using this form

What is true about the following code?




```
Course c1 = new Course("OOP",
        "Object-oriented programming");

Person p = new Person("Jens", 123);

Student s = (Student) p;

s.setEducation("Computer Science");

s.registerForCourse(c1);

System.out.println("Done");
```

a) The code doesn't compile
b) The code compiles but its execution never reaches the last statement
c) The code compiles and its execution completes without any problems
d) None of the above

A **typecast** allows for assigning an instance of the supertype to a variable with subtype type (downcasting)

```
T x = (V) y; // V must be T or a subtype of T
```

The code compiles, but because the ***runtime type*** of p is not Student (or a subclass of Student) a `java.lang.ClassCastException` is thrown during execution

```
// y's runtime type must be V or a subtype of V
T x = (V) y;
```

instanceof

```
Course c1 = new Course("OOP",
        "Object-oriented programming");

Person p = ...

p.setEmail("jens24@stu.unix.dk");

String welcomeMsg = "Welcome "+p.getName()+"!";

if (p instanceof Student) {

    Student s = (Student) p;

    s.setEducation("Computer Science");

    s.registerForCourse(c1);

}
```

Compile-time check:
- can an instance of **Person** be an instance of **Student**?

Runtime check:
- Is the runtime type of p **Student** (or a subclass of **Student**)?

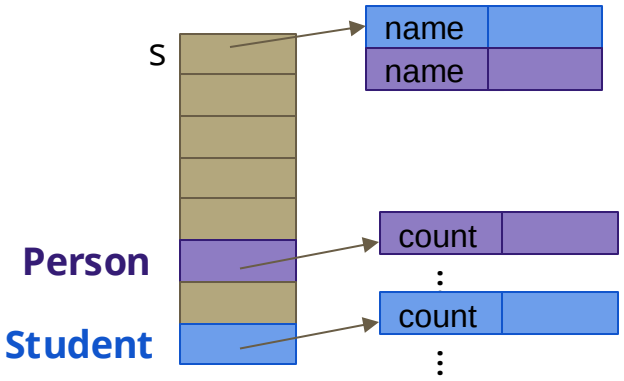
If we replace "`p instanceof Student`" by "`c1 instanceof Student`",
- Does the compile-time check pass?
- What value does it have when the code is executed?

Binding

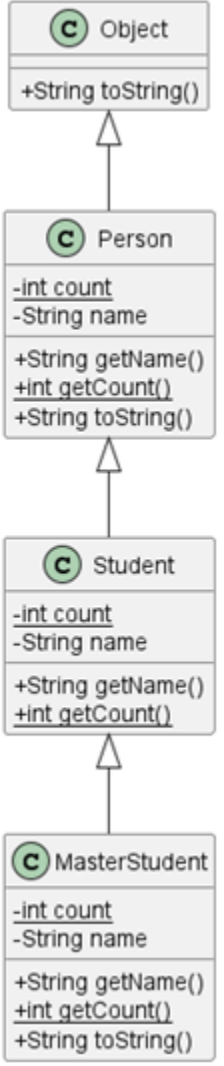
Binding is the process of identifying the accessed field, method's signature, method's code

- Binding starts at compile time, e.g., binding signatures of methods
- Some decisions are taken at runtime

```
Student s = new Student("Jens");
```



Student has two instance fields **name** and **name**, it inherits **getName**, **getCount**, **toString** from **Person** and provides its own definitions of **getName** and **getCount**



Underlined members in the class diagram represent **static** members

Signature: method name, number of parameters (arity), type of the parameters, order of the parameters

Binding

For fields, static methods, or non-static final methods:

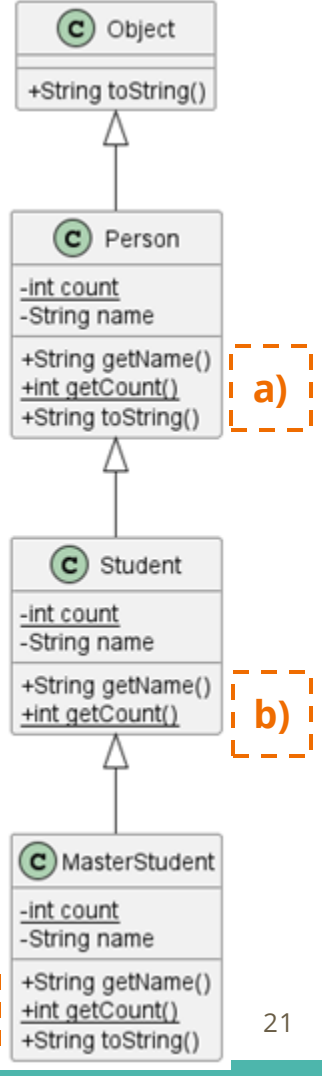
- it is a **compile-time** decision
- It is **early** binding (also known as static or compile time binding)

```
Student s = ...

String sStr = s.toString();

String name1 = s.getName();

int count1 = Student.getCount();

int count2 = s.getCount();

int count3 = ((Person) s).getCount();
```

?

For static members it is better to use the class as context (e.g., Student instead of s)

Which code would be executed in the last statement?



Please provide your answers using this form

Binding

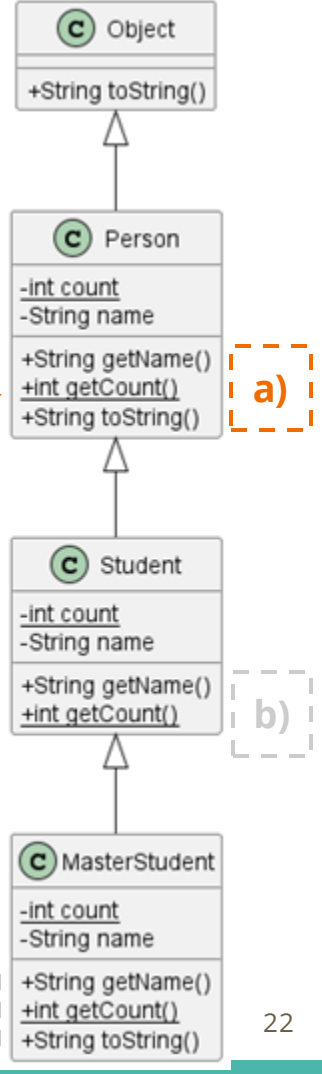
For fields, static methods, or non-static final methods:

- it is a **compile-time** decision
- It is **early** binding (also known as static or compile time binding)

```
Student s = ...

String sStr = s.toString();

String name1 = s.getName();

int count1 = Student.getCount();

int count2 = s.getCount();

int count3 = ((Person) s).getCount();
```

Which code would be executed in the last statement?

For static members it is better to use the class as context (e.g., Student instead of s)



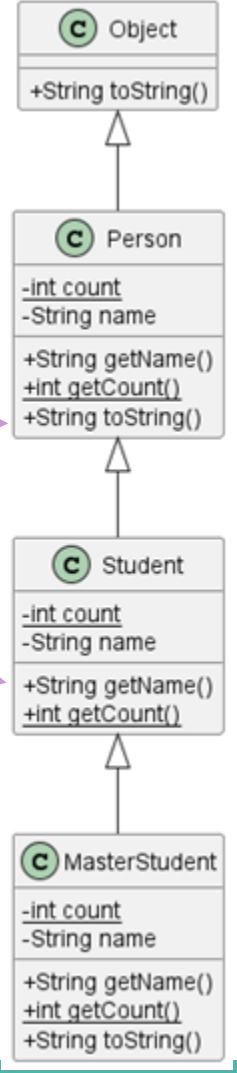
Binding

For non-static non-final methods:

- it is a **runtime** decision
- It is **late** binding (also known as dynamic or runtime binding)

```
Student s = new Student(...);

String sStr = s.toString();

String name1 = s.getName();

int count1 = Student.getCount();

int count2 = s.getCount();

String name2 = ((Person) s).getCount();
```

Which code would be executed?



Binding

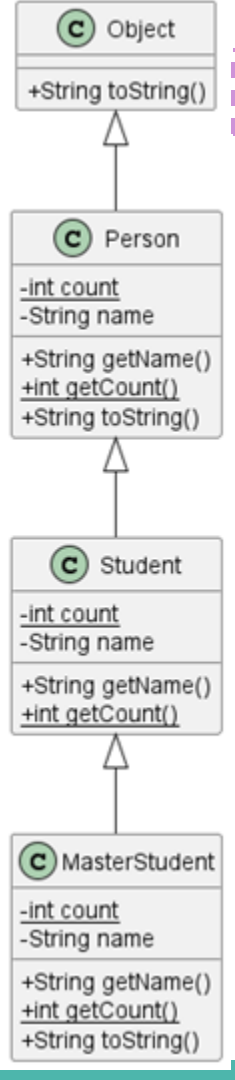
For non-static non-final methods:

- it is a **runtime** decision
- It is **late** binding (also known as dynamic or runtime binding)

```
Student s = new MasterStudent(...);

String sStr = s.toString();

String name1 = s.getName();

int count1 = Student.getCount();

int count2 = s.getCount();

String name2 = ((Person) s).getCount();
```

Which code would be executed in the second statement?

Please provide your answers using this form



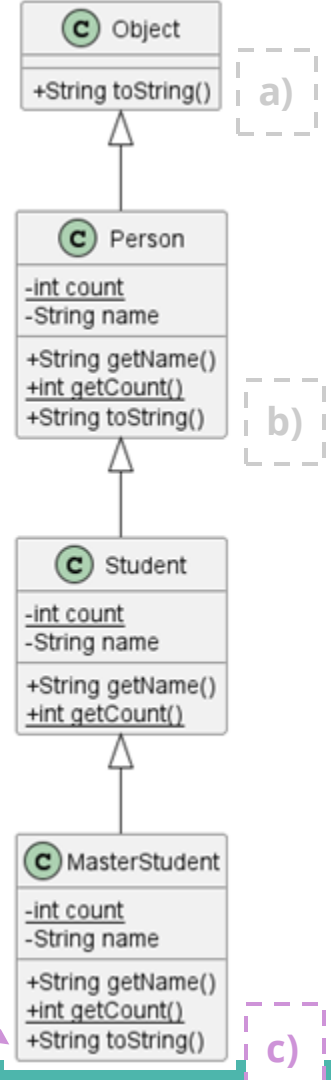
Binding

For non-static non-final methods:

- it is a **runtime** decision
- It is **late** binding (also known as dynamic or runtime binding)

```
Student s = new MasterStudent(...);

String sStr = s.toString();

String name1 = s.getName();

int count1 = Student.getCount();

int count2 = s.getCount();

String name2 = ((Person) s).getCount();
```

Which code would be executed in the second statement?



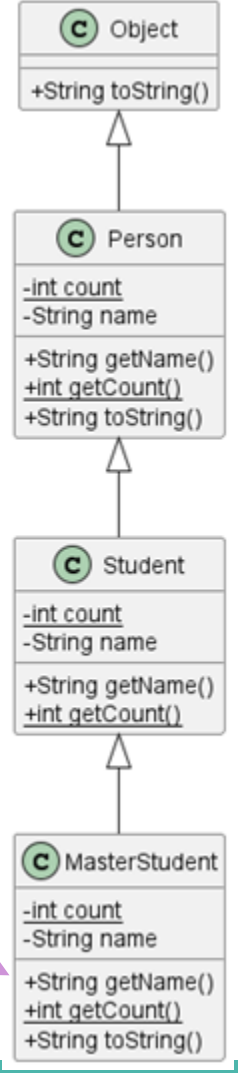
Binding

For non-static non-final methods:

- it is a **runtime** decision
- It is **late** binding (also known as dynamic or runtime binding)

```
Student s = new MasterStudent(...);

String sStr = s.toString();

String name1 = s.getName();

int count1 = Student.getCount();

int count2 = s.getCount();

String name2 = ((Person) s).getCount();
```

Which code would be executed in the third statement?

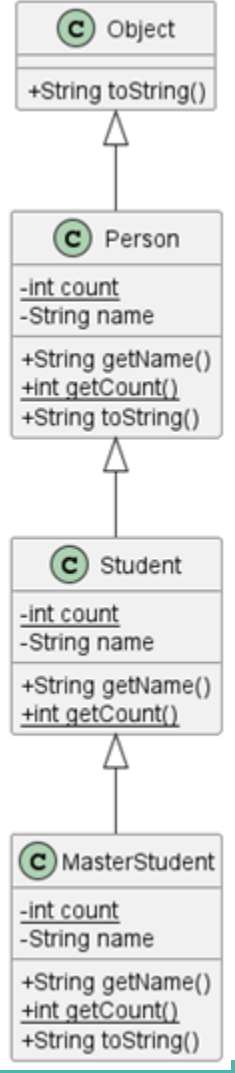


Method Overriding

A class C **overrides** an inherited non-static method by defining its own method with the **same signature** and ...

```
class Student extends Person {

    private String name;

    private static int count;

    ...

    @Override

    public String getName() {

        ...

    }
```

Student inherits the instance method **getName** from its superclass, but Student overrides the inherited method and provides its own definition of this method



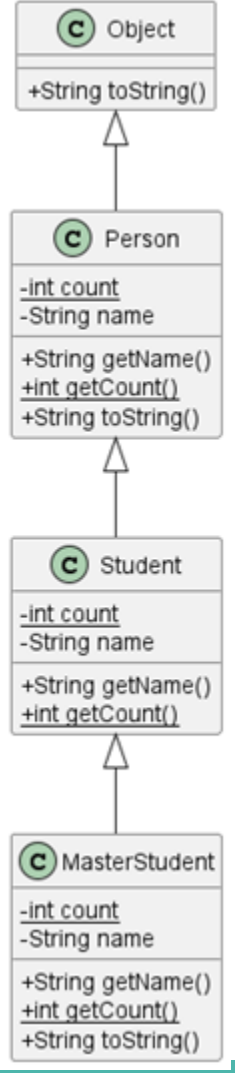
Because of **@Override**, the compiler checks that the method is overridden

Method Overriding

Is the method **toString** in **MasterStudent** overriding an inherited method?

a) No
b) Yes, it overrides the method inherited from Student
c) Yes, it overrides the method inherited from Person
d) Yes, it overrides the method inherited from Object

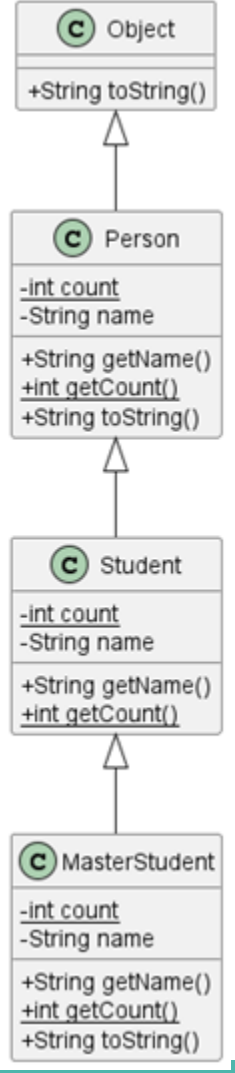
Please provide your answers using this form



Method Overriding

Is the method **toString** in **MasterStudent** overriding an inherited method?

a) No
b) Yes, it overrides the method inherited from Student
c) Yes, it overrides the method inherited from Person
d) Yes, it overrides the method inherited from Object



Method Overriding

```
class Student extends Person {
    private String name;
    private static int count;
    ...
    @Override
    public Student clone() {
        Student c = new Student(name);
        return c;
    }
}
```

A class C **overrides** an inherited non-static method by defining its own method with the **same signature** and

- an assignment-compatible return reference type
- the same return primitive type
- the same or more relax access level
- the same or less checked exceptions

Any instance of the subclass **must behave** as an instance of any of its ancestors

Student inherits the instance method **clone** from its superclass, but Student overrides the inherited method and provides its own definition of this method

Method Overriding

```
class Student extends Person {
    private String name;
    private static int count;
    ...
    @Override
    public Student clone() {
        Student c = new Student(name);
        return c;
    }
}
```

If Person **does not** provide its own definition of **clone**, what is the return type of the clone method inherited by Student?

a) Object
b) Person
c) Student
d) T

Please provide your answers using this form

Method Overriding

```
class Student extends Person {
    private String name;
    private static int count;
    ...
    @Override
    public Student clone() {
        Student c = new Student(name);
        return c;
    }
}
```

If Person **does not** provide its own definition of **clone**, what is the return type of the clone method inherited by Student?

a) Object
b) Person
c) Student
d) T

```
protected Object clone() throws CloneNotSupportedException
```

Accessing the superclass

```
class Student extends Person {
    private String name;
    private int studentId; ...
    public Student(String n, String stdId) {
        super(n);
        studentId = stdId;
    }
    ...
    @Override
    public String getName() {
        String n = super.getName();
        ...
    }
}
```

super is used to access the superclass' constructors and non-static methods and fields

If the constructor of the superclass is explicitly called, that **must be done as the first statement**. Otherwise the constructor of the superclass with no arguments is implicitly called

A default constructor with no arguments is provided only if no constructor is declared in that class

Method Overloading

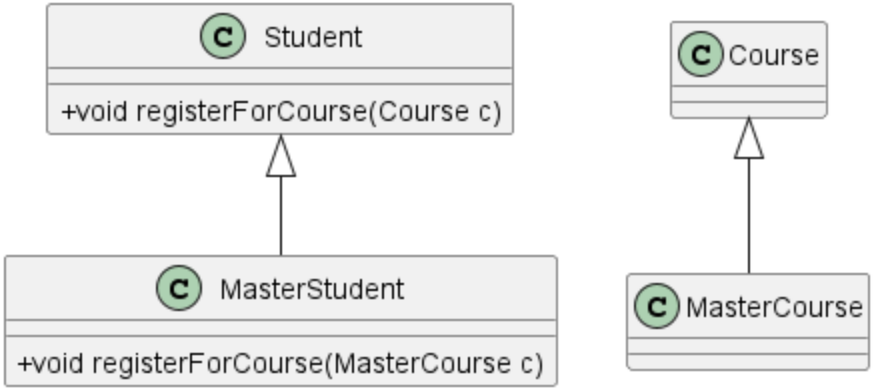
```
class Student extends Person {
    private int studentId; ...
    public String toString(boolean inclMeta) {
        String str = inclMeta ? "Student (" : "";
        str += inclMeta ? "name:" : "";
        str += super.getName() + ", ";
        str += inclMeta ? "id:" : "";
        str += studentId;
        str += inclMeta ? ")" : "";
        return str;
    }
}
```

A method is overloaded if there is more than one method with the **same name** but **different signature**

For an instance of Student, the method toString is overloaded:

- `toString(boolean)`
- `toString()`

Method Overloading



For an instance of MasterStudent, the method registerForCourse is overloaded:

- `registerForCourse(Course)`
- `registerForCourse(MasterCourse)`

For an overloaded method call, the compiler chooses **the most specific** method

```
MasterStudent s = ...

MasterCourse sp = ...

s.registerForCourse(sp);
```

`registerForCourse(MasterCourse)` is chosen

```
MasterStudent s = ...

Course c = new MasterCourse(...);

s.registerForCourse(c);
```

`registerForCourse(Course)` is chosen

Hiding

```
class Student extends Person {

    private String name;

    private static int count;

    ...

    public static int getCount() {

        ...

    }
```

A class C **hides** an inherited **static method** by defining its own method with the **same signature** and

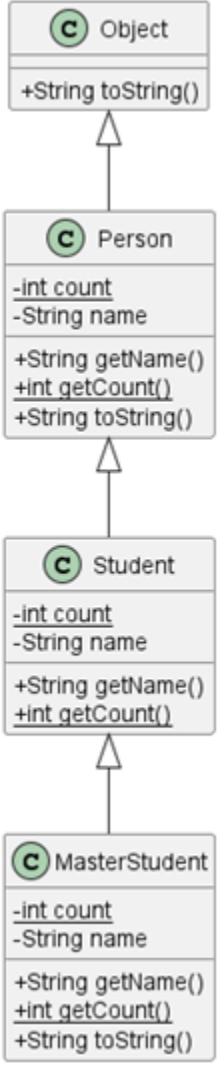
- an assignment-compatible return reference type
- the same return primitive type
- the same or more relax access level
- the same or less checked exceptions

Student inherits the static method **getCount** from its superclass, but the inherited method is **hidden** because **Student** provides its own definition of this method

~~**Student** inherits the field **name** from its superclass, but it is **hidden** by its own field with the same name~~

A class C **hides** an inherited **field** by defining its own field with the **same name**

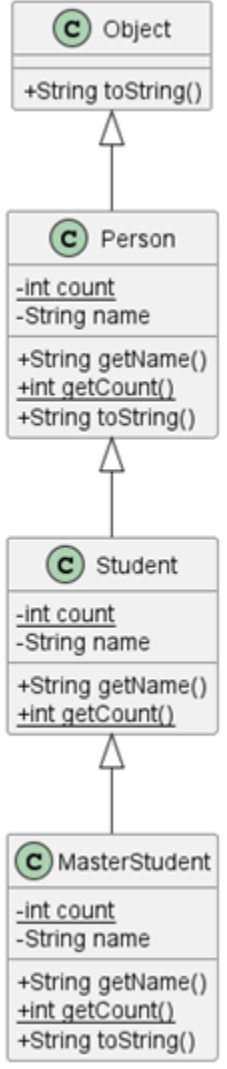
Since the fields are private, there are no inherited fields in the example



The following statement allocates memory for how many instance fields?

```
Student s = new MasterStudent(...);
```

a) 0
b) 1
c) 2
d) 3 or more

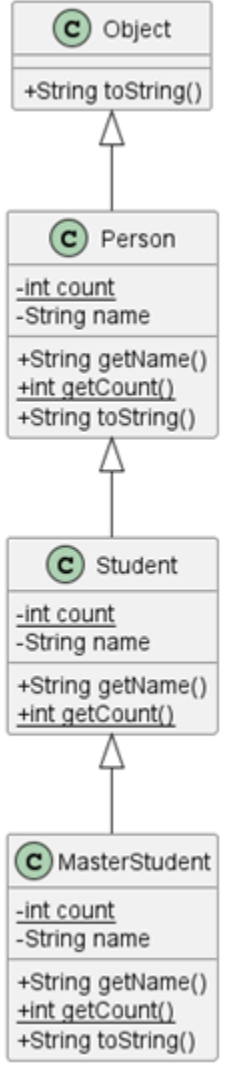


Please provide your answers using this form

The following statement allocates memory for how many instance fields?

```
Student s = new MasterStudent(...);
```

a) 0
b) 1
c) 2
d) 3 or more



An instance of MasterStudent is created. That instance has one instance field (name) and doesn't not inherit an instance field (name) from its superclass. Still memory is allocated for the instance fields defined in Student and Person, because they could be used through the inherited methods.

Can a class MasterStudent be a subclass of this class?

```
class Student extends Person {
    private String name;
    private int studentId; ...
    private Student(String n, String stdId) {
        super(n);
        studentId = stdId;
    }
    ...
    @Override
    public String getName() {
        String n = super.getName();
        ...
    }
}
```

Classes that cannot have any subclasses

```
final class Student extends Person {
    private String name;
    private int studentId; ...
    public Student(String n, String stdId) {
        super(n);
        studentId = stdId;
    }
    ...
    @Override
    public String getName() {
        String n = super.getName();
        ...
    }
}
```

final classes and classes with only private constructors cannot have any subclasses

Reason to disable inheritance of classes or overriding of methods: security, correctness, and performance

Abstract classes

- Represent concepts and cannot have instances

```
public abstract class Shape {
    private String name;
    public Shape(String name) {
        this.name = name;
    }
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public abstract double getPerimeter();
    public abstract double getArea();
}
```

- **Abstract methods** have only a declaration but no body
- If a class has **at least one abstract method**, it **must be declared as an abstract class**
- A class without any abstract methods can be declared as an abstract class
- It **has to be possible** to subclass an abstract class

Rectangle

```
public class Rectangle extends Shape {
    private double width;
    private double height;
    public Rectangle(double w, double h) {
        super("Rectangle");
        this.width = w;
        this.height = h;
    }
    @Override
    public double getPerimeter() {
        return 2.0 * (width + height);
    }
    @Override
    public double getArea() {
        return width * height;
    }
}
```

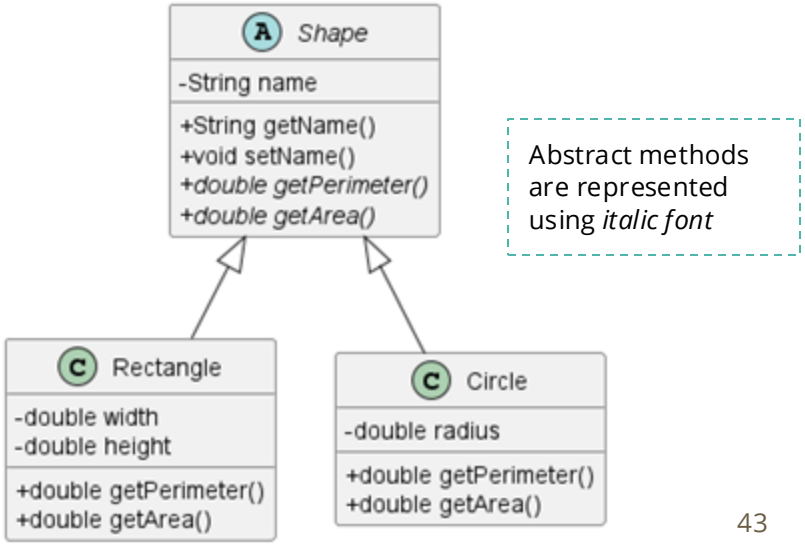
```
public abstract class Shape {
    private String name;
    public Shape(String name) {
        this.name = name;
    }
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public abstract double getPerimeter();
    public abstract double getArea();
}
```

- A concrete class is a class that is not an abstract class
- Concrete classes cannot have abstract methods
- A concrete class must override all abstract methods in its superclass

Circle

```
class Circle extends Shape {

    private double radius;

    public Circle (double radius) {
        super("Circle");
        this.radius = radius;
    }
    @Override
    public double getPerimeter() {
        return 2* Math.PI * radius;
    }
    @Override
    public double getArea() {
        return Math.PI * Math.pow(radius, 2);
    }
}
```

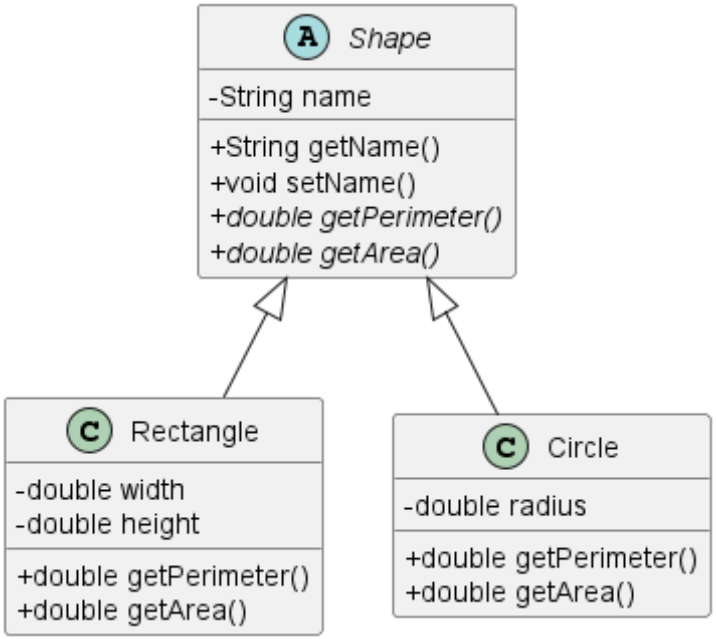
```
public abstract class Shape {
    private String name;
    public Shape(String name) {
        this.name = name;
    }
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public abstract double getPerimeter();
    public abstract double getArea();
}
```



Shape Example

```
Shape c = new Circle(3.0);
Shape r = new Rectangle(2.0, 4.0);
double d1 = c.getArea();
double d2 = r.getArea();
Shape[] shapes = { c, r };
double total = totalArea(shapes);
```

```
static double totalArea(Shape[] ss) {
    double area = 0.0;
    for (Shape s : ss) {
       area += s.getArea();
    }
    return area;
}
```



Generics

T is a **type variable**, it is implicitly bound as **T extends Object**

- Any type (that is subtype of Object)

```
public class Collection<T> {
  public boolean contains(T e) {
    ...
  }
  public <P extends Shape> getContained(P e) {
    ...
  }
}
```

Type **erasure**

```
public class Collection {
  public boolean contains(Object e) {
    ...
  }
  public getContained(Shape e) {
    ...
  }
}
```

Code with generic types

Code with raw types

For method overriding of generic classes/methods, the **same signature** check is done using the raw types (after type erasure)

Inheritance

- Inheritance facilitates **code reuse** and **customization**
- There must be a **“is-a” relationship** between subclass and superclass
- Subclasses **inherit** non-private members (fields / methods) of its superclass
- **Upcasting** is always possible, but **downcasting** is only possible in some cases
- **Early binding** is used for fields, static methods, and final methods
- **Late binding** is used for non-static non-final methods
- Subclasses can **override** inherited instance methods and **hide** inherited static methods and fields